

7. Multimodal Visual Understanding + SLAM Navigation + Visual Line Following

7. Multimodal Visual Understanding + SLAM Navigation + Visual Line Following

Introduction

Operation Steps

Preparatory Steps

Getting Started

Introduction

In this tutorial, we will learn how to issue commands via chat with Muto in the Dify interface.

Operation Steps

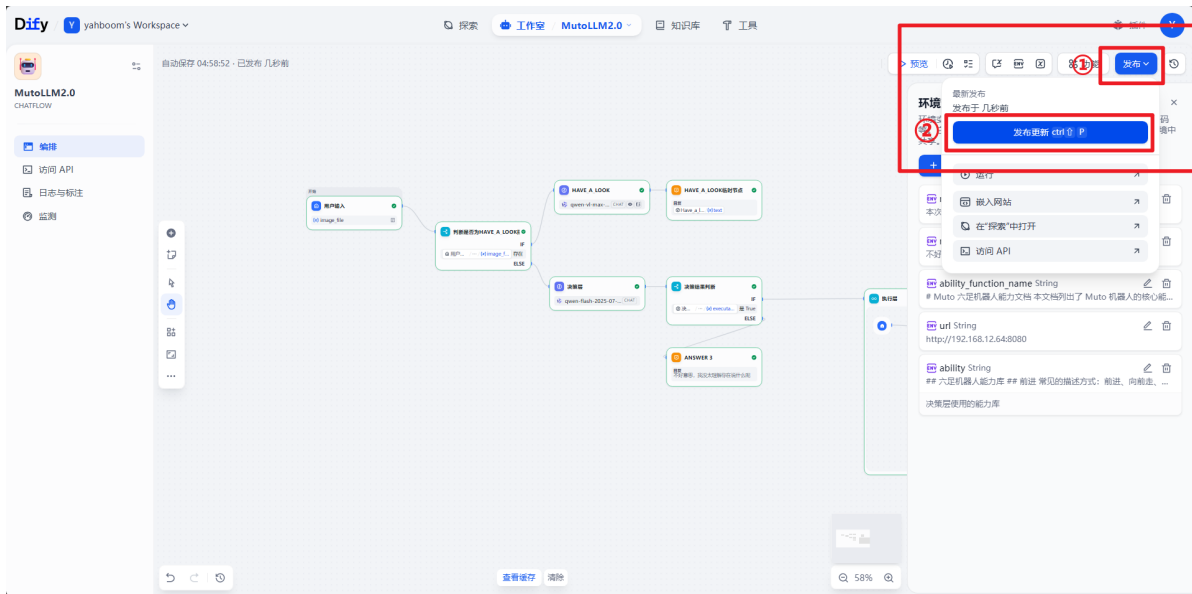
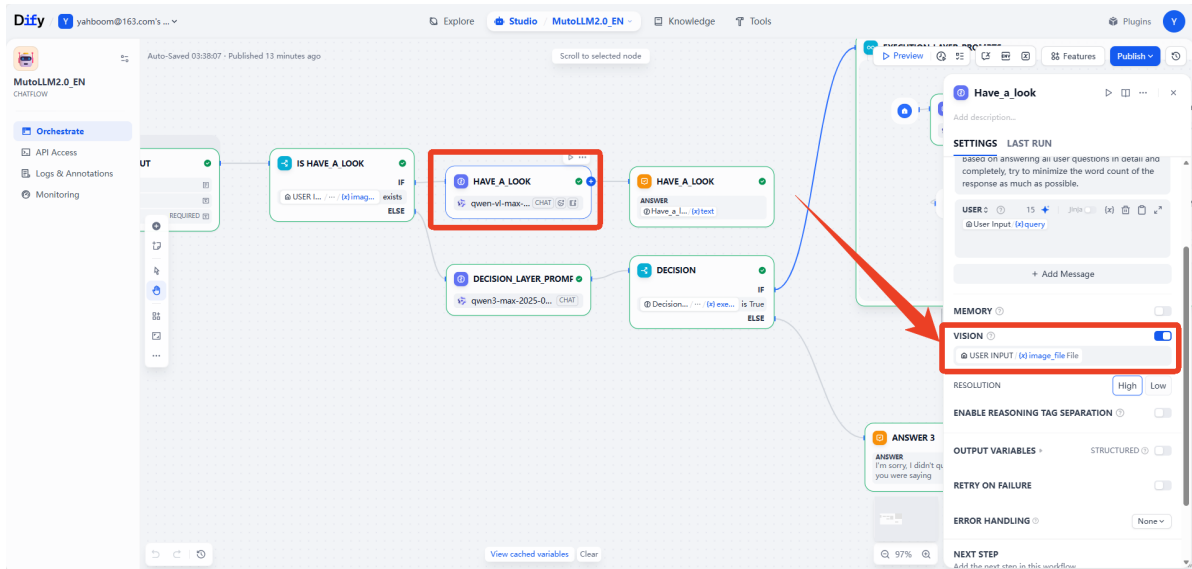
Preparatory Steps

This lesson requires that the Dify terminal be launched beforehand, and all configuration items have been configured as described in the previous section.

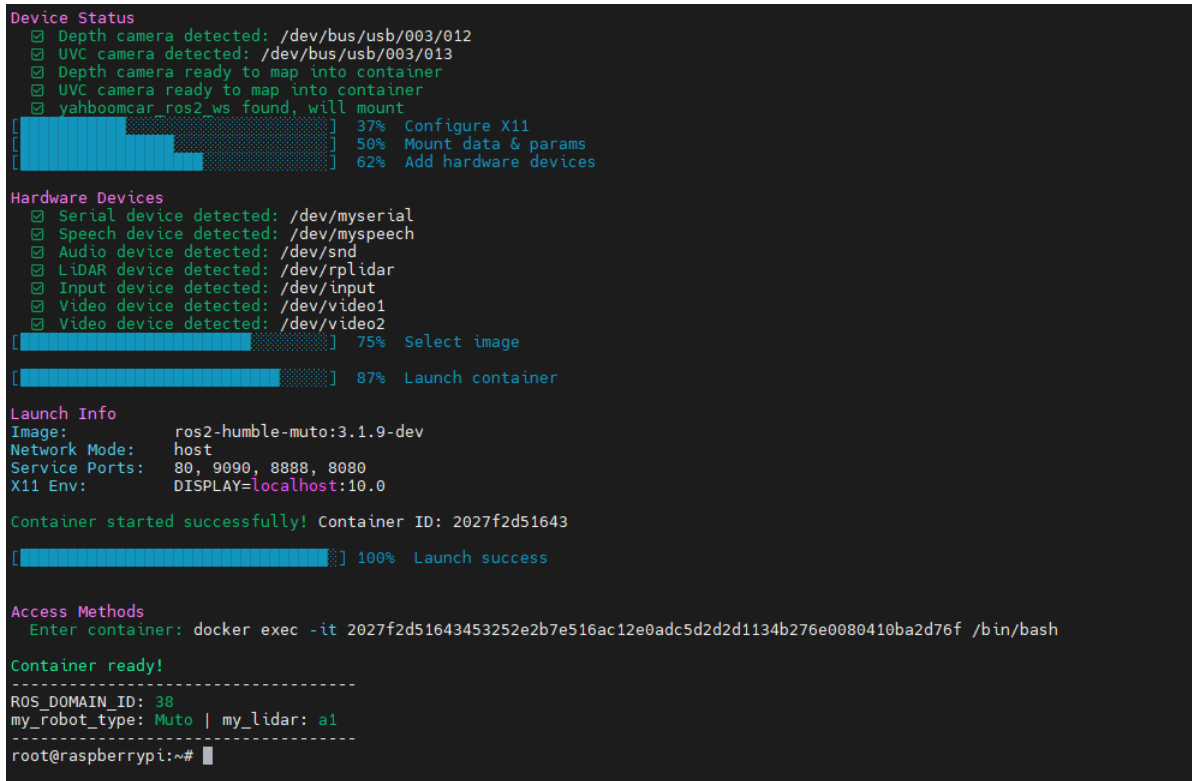
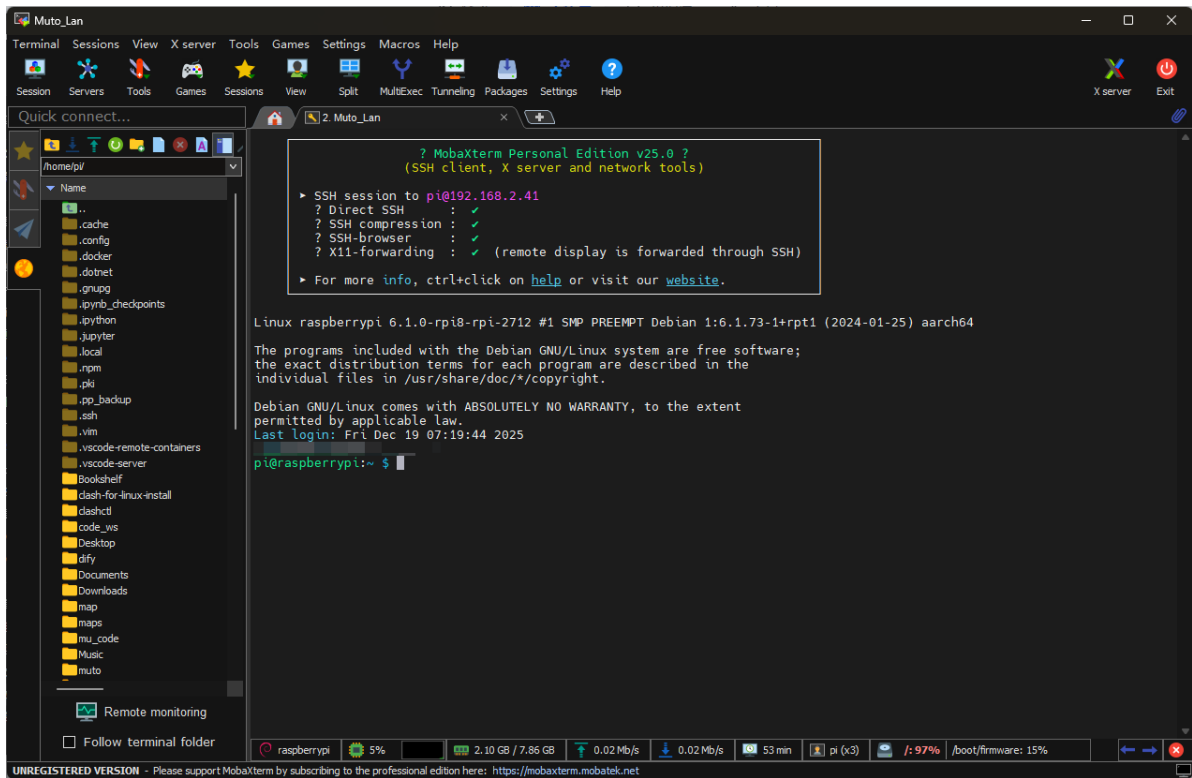
The image shows a terminal window and the Dify interface. The terminal window displays a script for configuring the environment. The script includes comments and commands for setting up the voice module, provider keys, and Dify configuration. Two specific lines are highlighted with red boxes and numbered 1 and 2:

```
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"
12
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 # export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
21 # Fallback if wlan0 has no IP
22 if [ -z "$DIFY_HOST" ]; then
23     export DIFY_HOST="127.0.0.1"
24 fi
```

The Dify interface shows a list of applications. The application "MutoLLM2.0 EN" is highlighted with a red box. The interface also shows a search bar and a list of tags.



This lesson requires MutoRS to be powered on. Follow the steps outlined in the previous chapters to start the large model terminal within Docker.



In this tutorial, we'll use a plain text version of the large model, starting it with the `--no-voice` parameter. Using `--no-voice` eliminates the risk of being interrupted by external voice prompts during use!

```
cd muto-llm-2.0/ && ./rebuild_and_launch.sh --no-voice
```

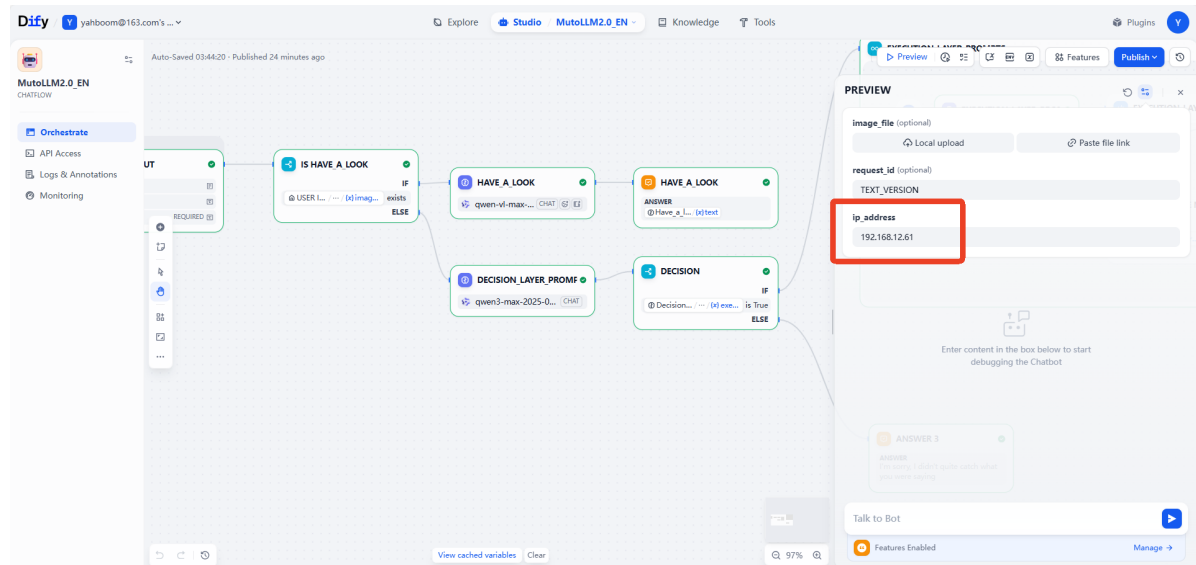
```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239218] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

Getting Started

For the large text model, you need to enter Muto's IP address here, which is the address used to access the Dify page.



In this section, we'll combine the navigation and line-following functions. Therefore, in addition to the large model's terminal program and Dify, we also need to open the navigation and line-following terminal programs.

Start the radar node:

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

Activate radar odometer node:

```
ros2 launch rf2o_laser_odometry rf2o_laser_odometry.launch.py
```

Start the navigation node:

```
ros2 launch hexapod_nav hexapod_navigation_mppi.launch.py autostart:=true  
map:=/root/yahboomcar_ros2_ws/maps/test_map.yaml
```

Please note that a pre-built map file (*.yaml and *.pgm; here we are using the map /root/yahboomcar_ros2_ws/maps/test_map) is required.

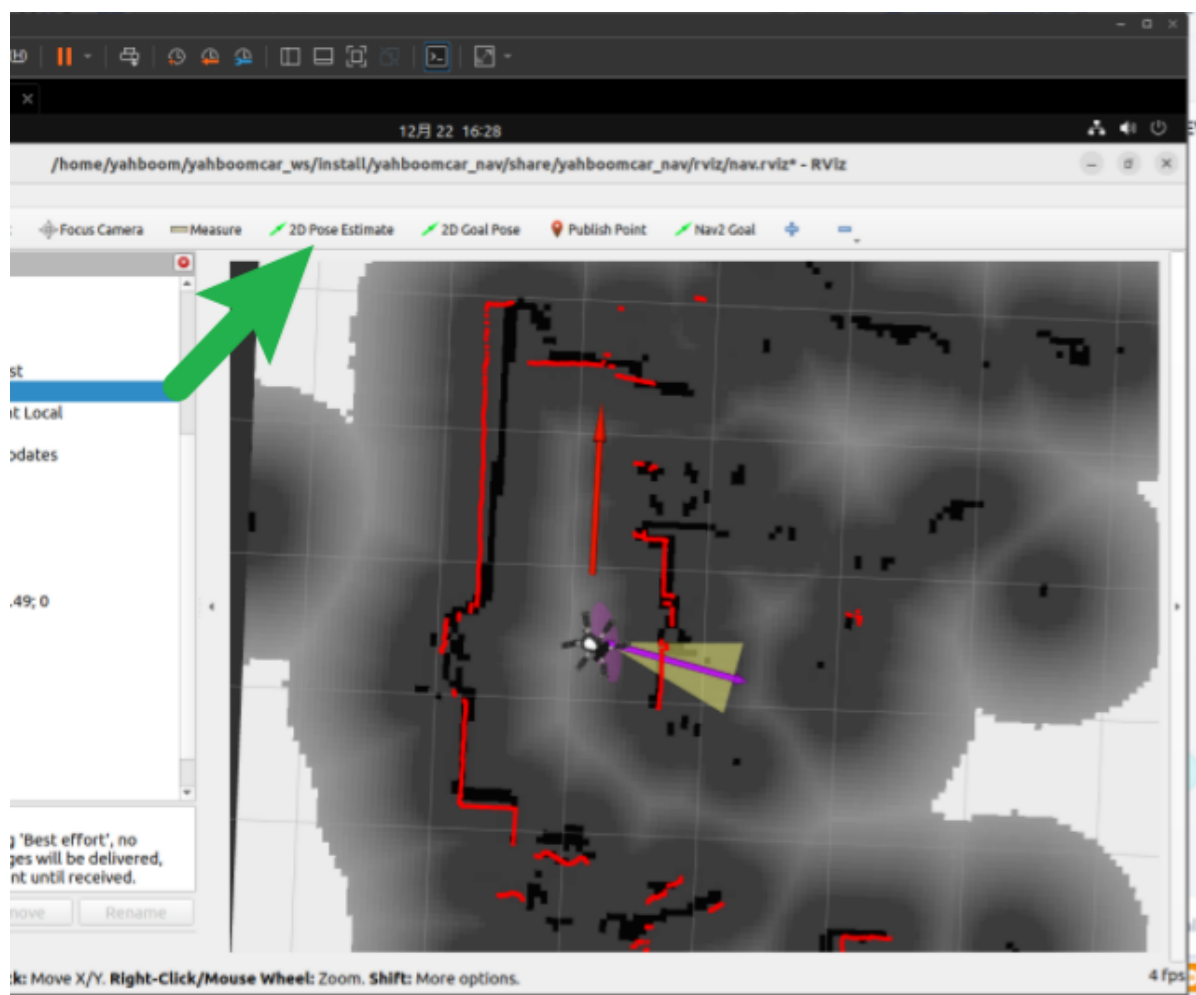
Finally, we can start the Rviz2 node to facilitate observation of the navigation process.

This step is recommended to be performed in a VMware virtual machine!!

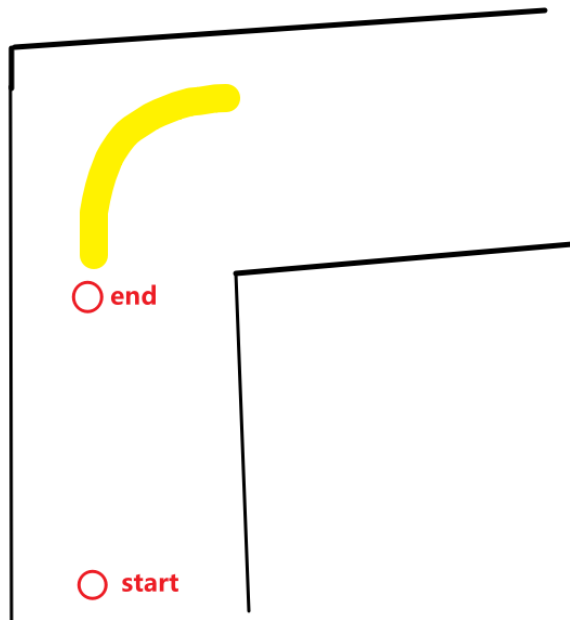
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Here, you must change the selected area in the image to Transient Local; otherwise, the map may not load.

After selecting it, we use the 2D Pose Estimate tool to set the initial position.



The scene where Muto is located is shown below:



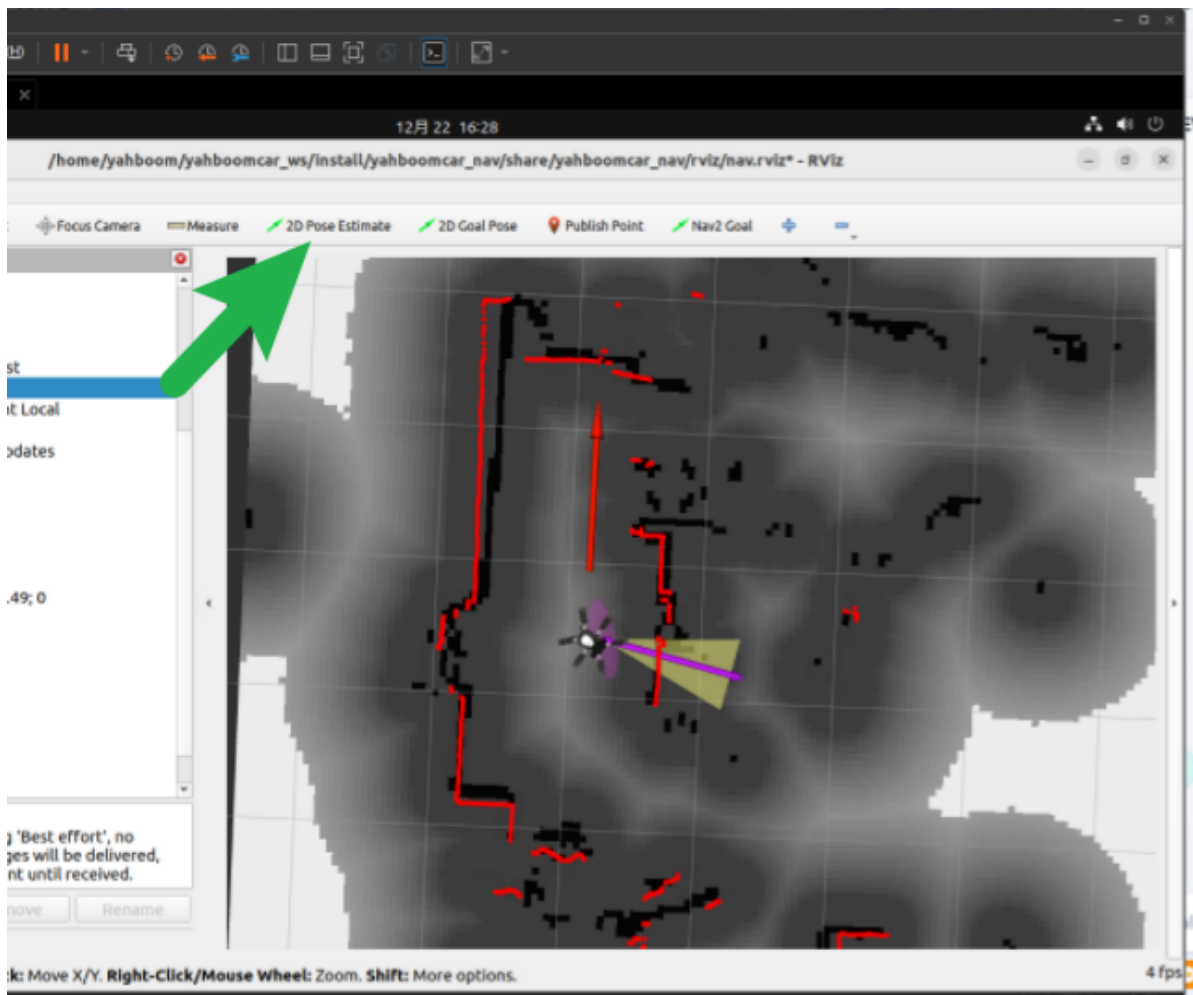
The yellow area is the line to be followed. The yellow line, and the red circles "end" and "start" represent two saved navigation points.

Our task is: "Let Muto navigate from any position to 'end,' then begin following the yellow line. After traversing the line, observe the surrounding environment, and finally navigate to the 'start' point."

Therefore, before starting, we need to record the HSV value of the yellow line (refer to previous chapters), and then record the two navigation points "start" and "end."

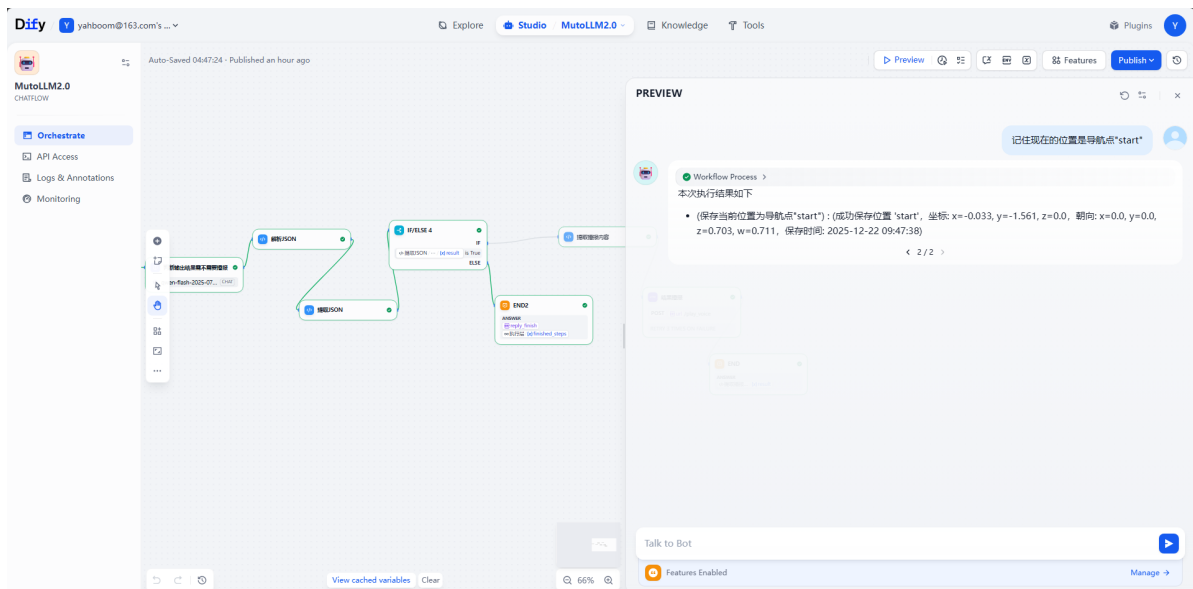
We first place Muto at the 'start' position.

Important! Before saving the navigation points, you need to set the initial position in Rviz using the 2D Pose Estimate tool.



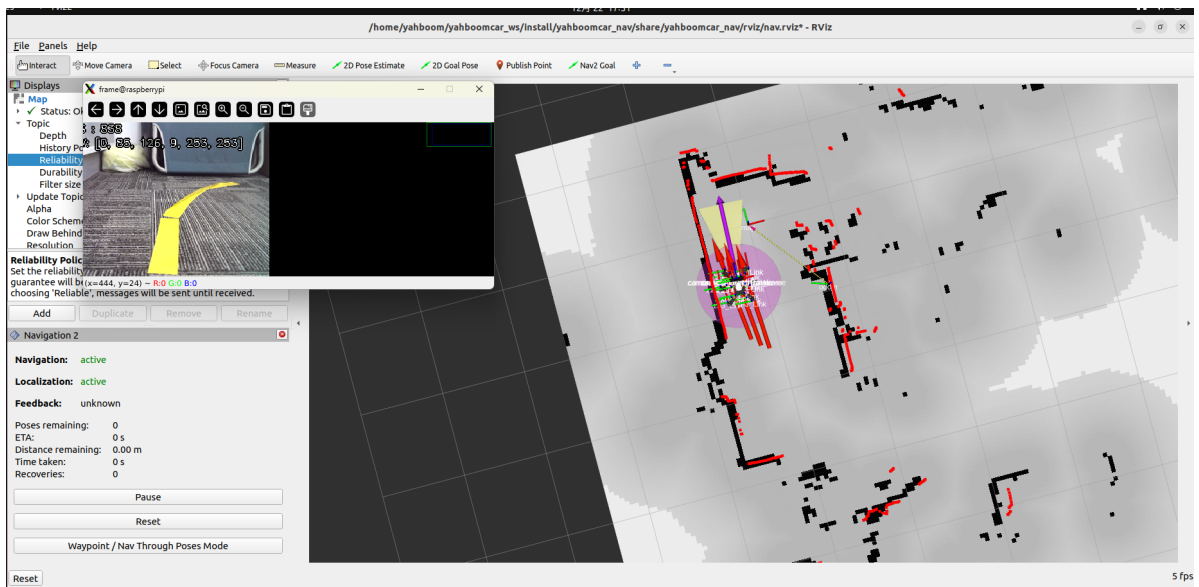
Then open the Divish terminal and enter:

Remember that the current position is the navigation point "start."



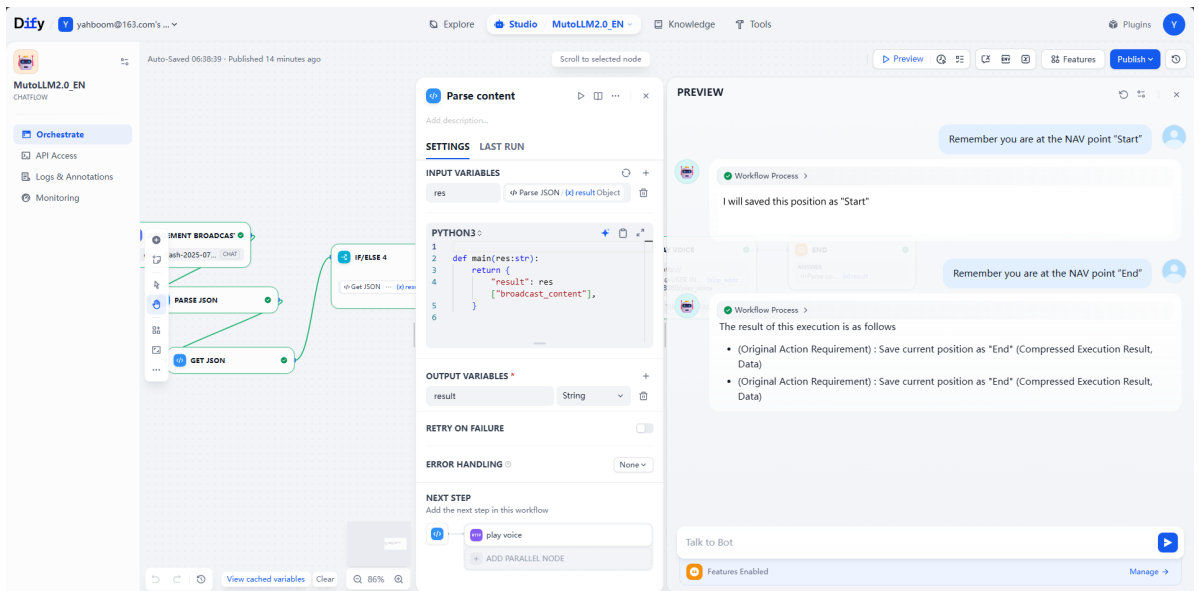
Next, we manually move Muto to the starting point of the traversal, "end."

After moving, it's best to recalibrate the position using the RViz tool to minimize coordinate errors.



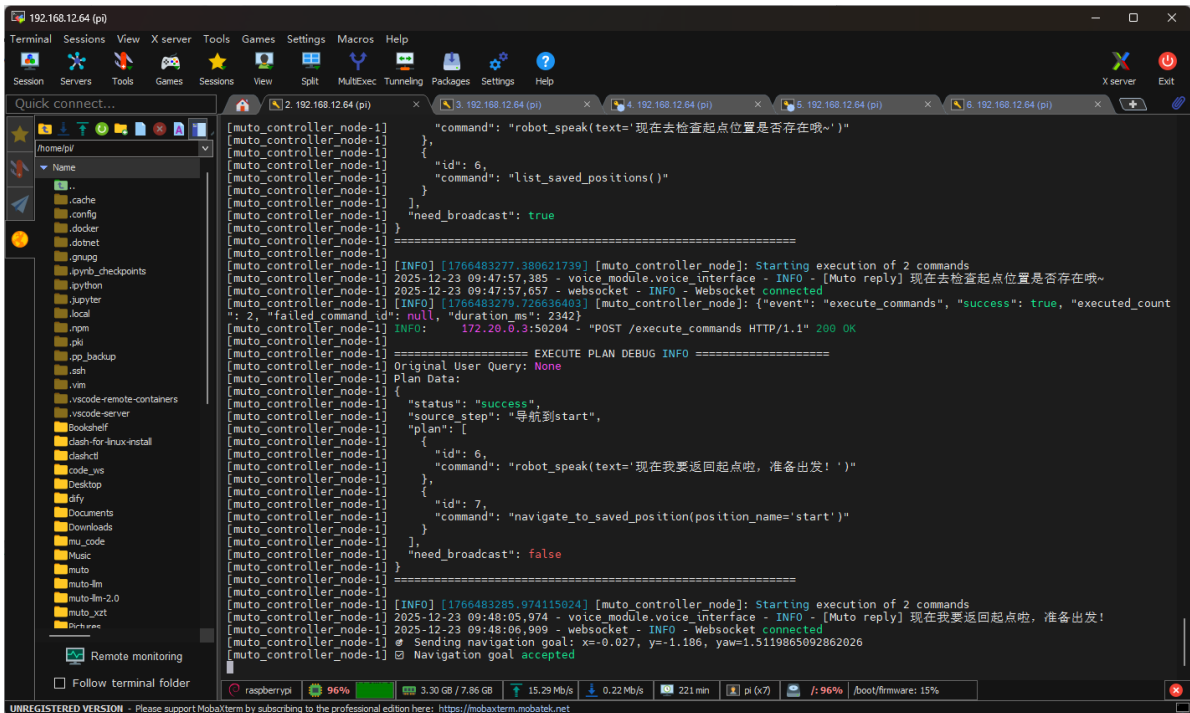
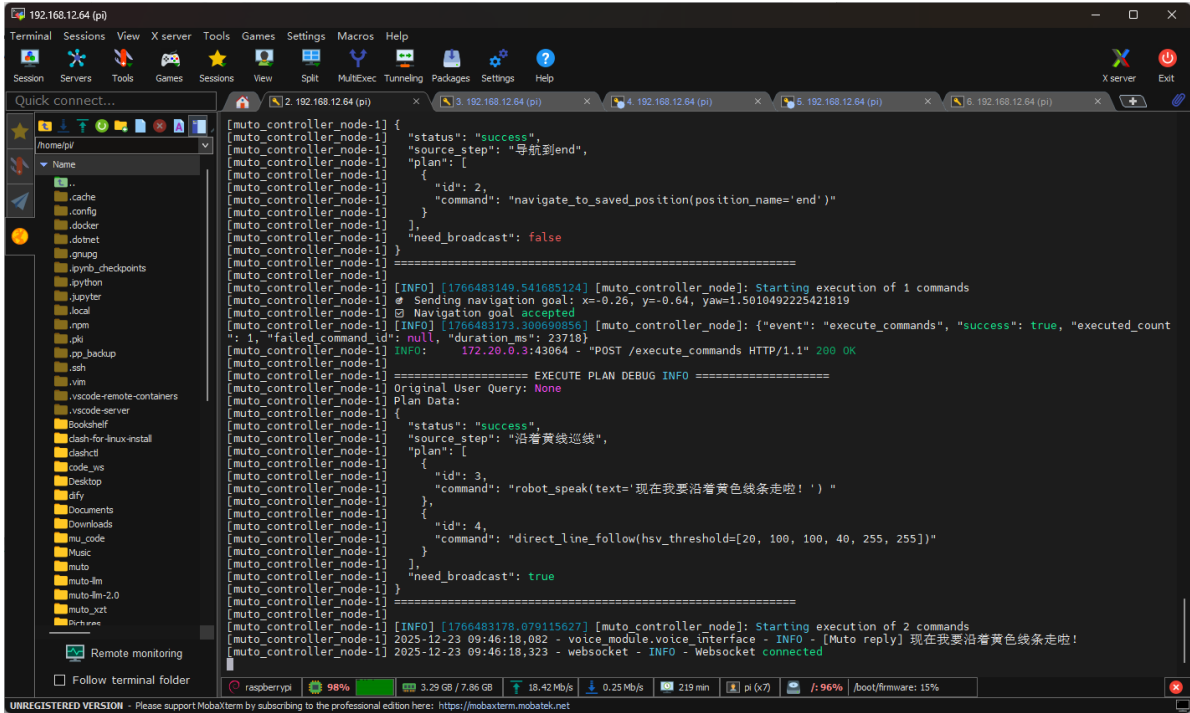
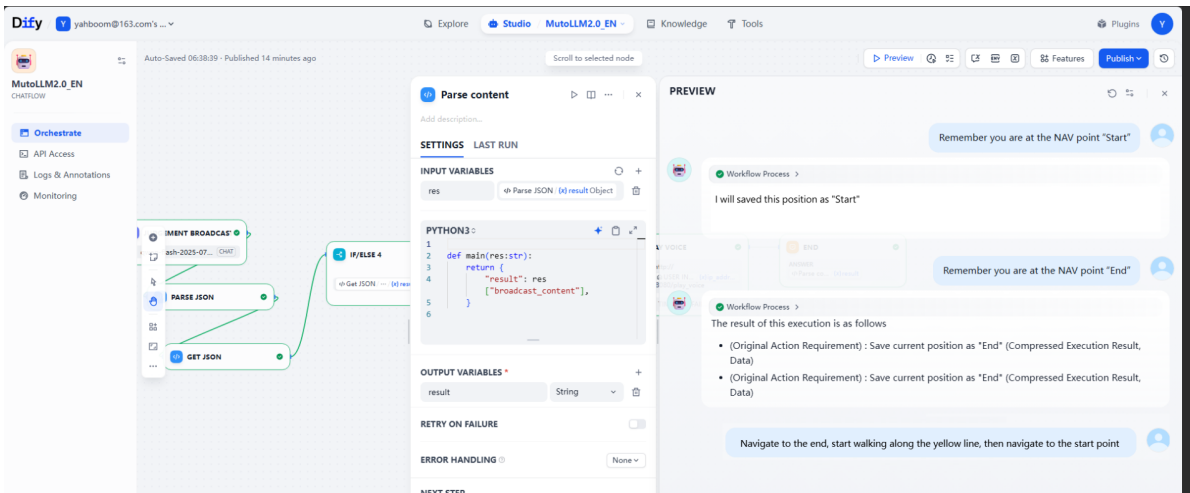
After moving, enter the following in the Dify terminal:

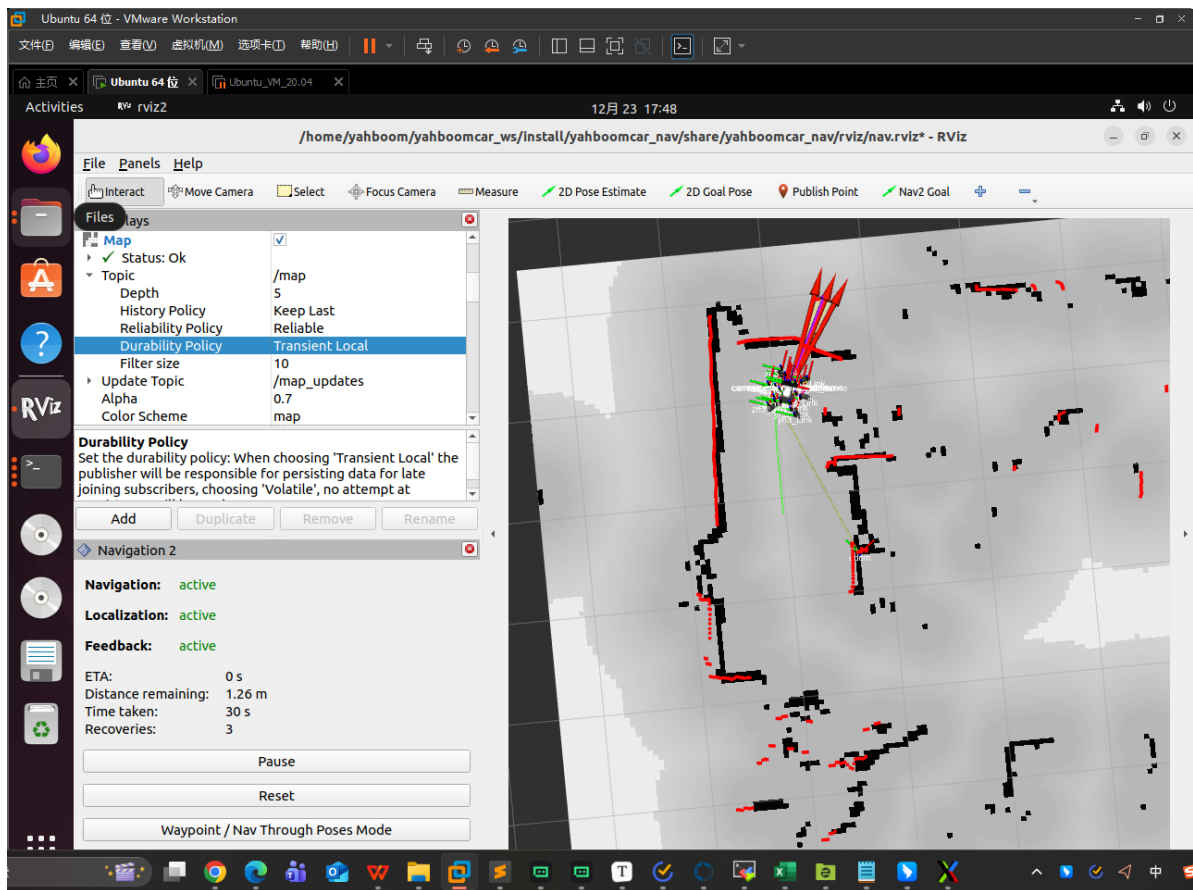
Remember that the current position is the navigation point "end"



Next, we move Muto to any position, then reconfirm the initial position using RViz, and then enter the following command in the Dify page:

Navigate to end, and then start following the yellow line. After completing the line, observe the surrounding environment, and then navigate back to the start point.





Finally, you can see that Muto has returned to the starting point.